

Занятие 12.

Статические члены класса. Указатель this. Виртуальные функции.

Задачи и упражнения общие для всех

1. Проверка д/з с прошлой пары.
2. Определить иерархию классов для данной предметной области:
корабль, пароход, парусник, корвет;
3. Определить в классе статическую компоненту – указатель на начало связанного списка объектов и статическую функцию для просмотра списка.
4. Реализовать классы. Компонентные данные класса специфицировать как protected. Статическую компоненту-данные инициализировать вне определения класса, в глобальной области. Для добавления объекта в список предусмотреть метод класса, т.е. объект сам добавляет себя в список. Например, `a.add()` – объект `a` добавляет себя в список. Включение объекта в список можно выполнять при создании объекта, т.е. поместить операторы включения в конструктор. В случае иерархии классов, включение объекта в список должен выполнять только конструктор базового класса. Вы должны продемонстрировать оба этих способа.
5. Написать демонстрационную программу, в которой создаются объекты различных классов и помещаются в список, после чего список просматривается.
6. Сделать соответствующие методы не виртуальными и посмотреть, что будет.
7. Реализовать вариант, когда объект добавляется в список при создании, т.е. в конструкторе.

Справочный материал

Статические члены класса должны быть определены в классе, как статические (static). Статические данные классов не дублируются при создании объектов, т.е. **каждый статический компонент существует в единственном экземпляре**.

Доступ к статическому компоненту возможен только после его инициализации. Для инициализации используется конструкция

тип имя_класса :: имя_данного инициализатор;

Например,

int complex :: count = 0;

Это предложение должно быть размещено в глобальной области после определения класса. Только при инициализации статическое данное класса получает память и становится доступным.

Обращаться к статическому данному класса можно обычным образом через имя объекта **имя_объекта.имя_компонента**

Но к статическим компонентам можно обращаться и тогда, когда объект класса еще не существует. Доступ к статическим компонентам возможен не только через имя объекта, но и через имя класса **имя_класса :: имя_компонента**

Однако так можно обращаться только к `public` компонентам. Для обращения к `private` статической компоненте извне можно с помощью **статических компонентов-функций**. Эти функции можно вызвать через имя класса.

имя_класса::имя_статической_функции

Пример

```
#include <iostream>
using namespace std;
class TPoint
{
    double x, y;
    static int N; // статический компонент □ данное : количество точек
public:
    TPoint(double x1 = 0.0, double y1 = 0.0) { N++; x = x1; y = y1; }
    static int& count() { return N; } // статический компонент-функция
};
int TPoint :: N = 0; //инициализация статического компонента данного
void main(void)
{
    setlocale(LC_ALL, "");
    TPoint A(1.0, 2.0);
    TPoint B(4.0, 5.0);
    TPoint C(7.0, 8.0);
    cout << "\nОпределены " << TPoint::count() << " точки\n";
}
```

Указатель this. Когда функция-член класса вызывается для обработки данных конкретного объекта, этой функции автоматически и неявно передается указатель на тот объект, для которого функция вызвана.

Этот указатель имеет имя **this** и неявно определен в каждой функции класса следующим образом:

имя_класса* `const this` = адрес_объекта

Указатель `this` является дополнительным скрытым параметром каждой нестатической компонентной функции. При входе в тело принадлежащей классу функции `this` инициализируется значением адреса того объекта, для которого вызвана функция. В результате этого объект становится доступным внутри этой функции.

В большинстве случаев использование `this` является неявным. В частности, каждое обращение к нестатической функции-члену класса неявно использует `this` для доступа к члену соответствующего объекта. Примером широко распространенного явного использования `this` являются операции со связанными списками.

Виртуальные функции. К механизму виртуальных функций обращаются в тех случаях, когда в каждом производном классе требуется свой вариант некоторой компонентной функции. Классы, включающие такие функции, называются полиморфными и играют особую роль в ООП. Виртуальные функции предоставляют механизм позднего (отложенного) или динамического связывания. Любая нестатическая функция базового класса может быть сделана виртуальной, для чего используется ключевое слово **virtual**.

```
class base
{
public:
    virtual void print() { cout << "\nbase"; }
    . . .
};
```

```
class dir : public base
{
public:
    void print() { cout << "\ndir"; }
};
void main()
{
    base B, * bp = &B;
    dir D, * dp = &D;
    base* p = &D;
    bp -> print(); // base
    dp -> print(); // dir
    p -> print(); // dir
}
```